Learning Successor Measures from Manipulation Play

Anonymous authors
Paper under double-blind review

Abstract

We study categorical successor densities for offline zero-shot reinforcement learning on object manipulation. The model rescales the discounted successor measure to a probability distribution and fits it with a softmax-normalized two-tower representation and a cross-entropy Bellman update; the hard variant of this update reduces to the categorical Bellman update underlying TD InfoNCE, while the soft variant learns a kernel-smoothed successor distribution over goal-relevant outcome coordinates. On dwell-heavy manipulation play, where the standard forward–backward (FB) estimator collapses, this model works: on OGBench cube-double, goal-conditioned success rises from 0.005 with plain FB to 0.736, and the gain survives exchanging the flow actor for DDPG+BC. Reusing the model's backward features as FB-style latent task prompts, however, fails at the behavior-cloning floor, and we trace this to an overlooked property of normalized successor models: the softmax leaves an additive gauge direction in the backward features unconstrained, and in practice this direction dominates them — its norm is 33–103× the centered signal — so point and reward-moment prompts collapse onto a task-irrelevant mean and reward-to-index inference performs at chance. We construct a centered dual task encoder, invariant to feature translations and positive affine reward transformations, and pair it with policy extraction that integrates reward samples directly against the predicted occupancy. In an initial single-seed experiment this restores latent-conditioned control from the behavior-cloning floor (0.111) to 0.812 on cube-single, about 87% of the goal-conditioned anchor. We state explicitly what remains open: estimator-level controls against TD InfoNCE, isolation of the privileged outcome coordinates, and validation of the general reward-sample interface.

1 Setting

The successor measure of a policy π is its discounted future-state occupancy,

$$M^\pi(s,a,X) = \mathbb{E}_\pi\left[\sum_{t\geq 0}\gamma^t \mathbf{1}[s_{t+1}\in X] \,\middle|\, s_0=s, a_0=a\right], \qquad Q_r^\pi(s,a) = \int r(x)\, dM^\pi(s,a,x), \tag{1}$$

so one learned measure evaluates any downstream reward (Dayan, 1993; Blier et al., 2021).

Forward-backward (FB) representations (Touati & Ollivier, 2021; Touati et al., 2023) factorize the density of this measure against the dataset marginal ρ,

$$m_z(s,a,x) = F(s,a,z)^\top B(x), \tag{2}$$

where F represents the current state and action and B represents a candidate future state. FB embeds a reward as

$$z_r \propto \mathbb{E}_{x\sim\rho}[r(x)B(x)] \tag{3}$$

and extracts values as

$$Q_r(s,a) \approx F(s,a,z_r)^\top z_r. \tag{4}$$

The standard FB estimator is

$$\mathcal{L}_{\mathrm{FB}} = \tfrac{1}{2}\mathbb{E}_{x\sim\rho}\left[\left(m_z(s,a,x) - \gamma\bar{m}_z(s^+,a^+,x)\right)^2\right] - \mathbb{E}\left[m_z(s,a,s^+) - \gamma\bar{m}_z(s^+,a^+,s^+)\right] + \lambda_{\mathrm{ortho}}\mathcal{L}_{\mathrm{ortho}}, \tag{5}$$

where a^+ is sampled from the policy associated with z. Random future-state queries receive the squared Bellman residual, while the observed successor supplies the immediate positive term.

This paper sits in two active lines of work. Normalized and generative successor measures have several precedents: temporal-difference flows (Farebrother et al., 2025), Bellman diffusion models (Shribak et al., 2024), and intention-conditioned occupancy models (Zheng et al., 2026b) all represent the successor measure as a probability distribution, and Section 3.5 relates our estimator to the closest precedent, TD InfoNCE (Zheng et al., 2024). The FB lineage has likewise moved past the original formulation: conservatism for low-quality data (Jeen et al., 2024), autoregressive task features with advantage-weighted extraction (Cetin et al., 2024), dual-value variants (Sun et al., 2025), stochastic policies and general utilities (Bagatella et al., 2026b), critiques of joint FB training (Zheng et al., 2026a), distribution-corrected off-policy variants (Asadulaev et al., 2026), latent-dynamics pretraining (Bagatella et al., 2026a), and reward-sample task encoders (Frans et al., 2024). Task inference itself has prior art: covariance-corrected reward regression is the standard successor-feature encoding (Cetin et al., 2024), and OpTI-BFM (Rupf et al., 2026) formalizes behavior-foundation-model task inference as exactly this regression, studied under online reward feedback. Our contribution on that axis is not the regression but the reason it becomes mandatory here: the density objective leaves the backward features identifiable only up to transformations the softmax cannot see, so they are not automatically a valid task basis (Section 6), and the regression must moreover be the centered one. We position against these where each becomes relevant.

Concretely, this paper contributes:

1. Empirical: a documented failure of the standard LSIF-style FB estimator on OGBench manipulation play, and a categorical successor-density model that works there ($0.005 \to 0.736$ on cube-double), with the gain surviving a flow-to-DDPG+BC actor swap (Section 4).

2. Methodological: outcome-space, task-conditioned categorical successor densities with offline, reward-integrating policy extraction, including the soft variant and its kernel-smoothed fixed point (Section 3).

3. Analytical: a gauge analysis of latent task prompting in normalized successor models: the softmax removes FB's linear reward-to-index identity and leaves an additive gauge direction unconstrained, and we measure that gauge dominating the learned features and breaking every naive prompt built from them (Section 6).

4. Algorithmic: a centered dual task encoder — invariant to feature translations and positive affine reward transformations — combined with external reward-sample integration, validated so far in an initial single-seed screen (Section 6, with the outstanding checks in Section 7).

The first two are demonstrated; the fourth is preliminary. We do not claim the estimator itself, which reduces to TD InfoNCE's update (Proposition 1; Zheng et al., 2024); nor normalized successor measures as objects (Farebrother et al., 2025; Shribak et al., 2024; Zheng et al., 2026b); nor covariance-corrected reward regression per se (Cetin et al., 2024; Rupf et al., 2026). The contribution is where these meet: a normalized successor model whose feature gauge silently breaks the task interface it inherits from FB, and the corrected interface that repairs it.

2 Why manipulation play stresses the FB estimator

To see the problem, it helps to separate the two parts of Equation 5. For a random future-state query $x \sim \rho$, the quadratic term

$$\frac{1}{2}\mathbb{E}_{x\sim\rho}\left[\left(m_z(s,a,x) - \gamma\bar{m}_z(s^+,a^+,x)\right)^2\right] \tag{6}$$

fits the bootstrapped successor density. The second term supplies the observed one-step successor,

$$-\mathbb{E}\left[m_z(s,a,s^+) - \gamma\bar{m}_z(s^+,a^+,s^+)\right]. \tag{7}$$

Thus every transition gives a positive update at its realized successor.

This becomes problematic in manipulation play. The robot may continue acting for many steps while the goal-relevant object (e.g. the cube) does not move: approach, grasp, transport, release, then rest. On cube-single-play-v0, the cube is stationary on 48% of transitions and stationary runs have median length 44 steps. On cube-double, each cube is stationary on 72% of transitions; on scene, the drawer and window are stationary on 91% and the buttons on 99.6%.

Consider a stationary run of length L. The same object configuration appears as the observed successor on all L transitions, so the FB objective receives L nearly identical positive updates there. The transition that moved the object into that configuration appears only once. At the median run length of 44 steps, the stationary configuration therefore influences the observed-successor term roughly forty times more often than the transition that brought the object there.

The successor measure should assign mass to these stationary visits. The problem is how FB represents that mass, specifically by predicting

$$m_z(s,a,x) = F(s,a,z)^\top B(x) \tag{8}$$

which is an unconstrained scalar density. The true successor measure, however, has fixed total mass

$$M^\pi(s,a,\mathcal{X}) = \sum_{t\geq 0}\gamma^t = \frac{1}{1-\gamma}. \tag{9}$$

This constraint is not enforced by the bilinear prediction itself. Repeated positive updates at persistent configurations can therefore be expressed by increasing critic scale rather than only by reallocating a fixed amount of future occupancy.

This is the imbalance we target, but its causal status should be stated carefully, because two facts cut against reading the count argument as a proven mechanism. First, the population FB objective has a finite scale optimum — scaling a candidate m by c changes the loss by $\frac{c^2}{2}\mathbb{E}_\rho[m^2] - c\,\mathbb{E}_{M^\pi}[m]$, minimized at finite c — and stationary configurations also appear proportionally more often as negative queries drawn from the data marginal, which pushes back on the same states. In our own estimator this counterweight is built in: the query support is the batch's next states, so dwell configurations enter the negatives exactly as often as they enter the positives. Second, the repetition argument counts configurations in the object coordinates $\phi(s')$, but the plain-FB baseline's backward map receives the full observation, in which the arm keeps moving while the object rests; the baseline's successor inputs during a stationary run are therefore not identical. We accordingly claim the empirical contrast — plain FB fails on these dwell-heavy datasets while the normalized categorical estimator below does not — and identify outcome imbalance and the unconstrained scale direction as candidate factors, without asserting a unique causal channel. A plain-FB control that receives the same $\phi(s')$ is future work (Section 7).

3 Density-FB: normalized successor learning

3.1 What is being predicted

Write (s,a,s^+) for a transition from the offline dataset, and define

$$x = \phi(s), \qquad x^+ = \phi(s^+) \tag{10}$$

as the state variables whose future occupancy we want to predict.

The forward and backward representations see different inputs:

$$F(s, a, g) \in \mathbb{R}^d, \qquad B(x) \in \mathbb{R}^d. \tag{11}$$

F receives the full current state and action, while B represents a candidate future configuration.

In our manipulation experiments, ϕ extracts the object coordinates. We therefore condition on the full state but predict future occupancy over the goal-relevant object configuration. This is the pushforward of the full-state successor measure, and it fixes the method's reward scope: any bounded reward measurable on the chosen outcome coordinates and supported by the data marginal — not arbitrary MDP rewards, which may depend on robot joint state, action costs, contact forces, or path-dependent quantities that ϕ discards. The choice of ϕ is also privileged information: the method receives the goal-relevant coordinate blocks and their tolerances ϵ_b, which the plain-FB baseline does not; no reward labels are used, but the pretraining is not domain-agnostic. Setting ϕ to the identity instead gives $x = s$ and asks B to represent the full state; we are testing this separately.

3.2 From a successor measure to a probability distribution

For a goal-conditioned policy $\pi_g(a \mid s)$, define the successor measure over x as

$$M^{\pi_g}(s, a, X) = \mathbb{E}_{\pi_g}\left[\sum_{t\geq 0} \gamma^t \mathbf{1}[x_{t+1} \in X] \;\middle|\; s_0 = s, a_0 = a\right]. \tag{12}$$

Its total mass is

$$M^{\pi_g}(s, a, \mathcal{X}) = \frac{1}{1-\gamma}, \tag{13}$$

independent of (s, a, g).

We therefore normalize it:

$$\nu^{\pi_g}(\cdot \mid s, a) = (1-\gamma) M^{\pi_g}(s, a, \cdot). \tag{14}$$

Now

$$\nu^{\pi_g}(\mathcal{X} \mid s, a) = 1. \tag{15}$$

This normalization only rescales the usual successor-measure Bellman equation.

$$M^{\pi_g}(\cdot \mid s, a) = P_\phi(\cdot \mid s, a) + \gamma\, \mathbb{E}_{s^+, a^+}\left[M^{\pi_g}(\cdot \mid s^+, a^+)\right], \tag{16}$$

$$\nu^{\pi_g}(\cdot \mid s, a) = (1-\gamma) P_\phi(\cdot \mid s, a) + \gamma\, \mathbb{E}_{s^+, a^+}\left[\nu^{\pi_g}(\cdot \mid s^+, a^+)\right], \tag{17}$$

where the second line follows by multiplying the first by $1-\gamma$ and using $\nu = (1-\gamma)M$. Thus the factor $1-\gamma$ appears only in the immediate term because we normalize the measure to unit mass; the recursive coefficient remains γ. Consequently, for any ν_1, ν_2, the immediate term cancels and

$$\|\mathcal{T}\nu_1 - \mathcal{T}\nu_2\|_{\mathrm{TV}} \leq \gamma \|\nu_1 - \nu_2\|_{\mathrm{TV}}, \tag{18}$$

so the normalized Bellman operator has the same γ-contraction as the original successor-measure operator.

3.3 Normalize the FB scores

For a minibatch $\{(s_i, a_i, s_i^+)\}_{i=1}^n$, we construct a query set from its observed successors (the cube positions)

$$U = \{u_j\}_{j=1}^n, \qquad u_j = \phi(s_j^+). \tag{19}$$

For transition i and candidate future configuration u_j, we compute

$$\ell_{ij} = \frac{F(s_i, a_i, g_i)^\top B(u_j)}{\tau}. \tag{20}$$

Standard FB interprets the bilinear quantity directly as a density. Density-FB instead treats it as a logit and normalizes across candidate successors:

$$p_{ij} = \frac{\exp(\ell_{ij})}{\sum_{k=1}^{n} \exp(\ell_{ik})}. \tag{21}$$

Hence

$$p_{ij} \geq 0, \qquad \sum_{j} p_{ij} = 1. \tag{22}$$

The vector p_i represents the normalized future occupancy $\nu^{\pi_{g_i}}(\cdot \mid s_i, a_i)$ on the sampled query set U. The query set is redrawn on every update, so its size is a Monte-Carlo sample size.

3.4 The hard Bellman target

Equation 17 gives the exact measure-level recursion. For a sampled transition, the observed x_i^+ provides a sample from the immediate distribution $P_\phi(\cdot \mid s_i, a_i)$. Because every batch successor is included in U, row i's observed successor occupies slot i.

Define the one-hot vector

$$e_{ij} = \mathbf{1}[j = i]\,. \tag{23}$$

Next, sample

$$a_i^+ \sim \pi_{\bar{\omega}}(\cdot \mid s_i^+, g_i) \tag{24}$$

and let $\bar{p}_i^+$ be the target-network prediction from (s_i^+, a_i^+, g_i) evaluated over the same query set U.

The sampled Bellman target is then

$$p_i^{\text{tgt}} = \text{sg}\left[(1-\gamma)e_i + \gamma \bar{p}_i^+\right]. \tag{25}$$

Both e_i and $\bar{p}_i^+$ have unit mass, so

$$\sum_{j} p_{ij}^{\text{tgt}} = (1-\gamma) + \gamma = 1. \tag{26}$$

We fit the predicted occupancy to this target with cross-entropy:

$$\mathcal{L}_{\text{DensityFB}} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{n} p_{ij}^{\text{tgt}} \log p_{ij}. \tag{27}$$

The update has a simple interpretation. The observed successor receives $1-\gamma$ of the normalized occupancy, while the remaining γ is inherited from the predicted occupancy after the next state. Equations 21–27 define hard Density-FB.

3.5 Relation to TD InfoNCE

The hard estimator is not new in isolation. Its target is the categorical Bellman update of TD InfoNCE (Zheng et al., 2024), which classifies the observed next state against negatives from the state marginal, bootstraps with the stop-gradient softmax of the next state-action's scores, and whose analysis shows the resulting classifier represents the normalized discounted future-state distribution.

Proposition 1 (Relation to TD InfoNCE). With the one-hot immediate label e_i, a query support shared between the immediate and bootstrap terms, and matched normalization conventions, the hard Density-FB loss (Equation 27) reduces to the categorical Bellman update underlying TD InfoNCE, evaluated on the outcome coordinates $x = \phi(s)$ and with the task variable carried in the forward tower.

We therefore do not claim the estimator as a contribution. The proposition is a reduction under matched conventions, not an implementation identity: TD InfoNCE labels the dynamics next state in its immediate term but a marginal draw in its bootstrap term (with one shared negative pool), uses ℓ_2-normalized towers, and extracts its policy with a goal-contrastive cross-entropy plus a goal-conditioned behavioral-cloning regularizer over relabeled goals. It is, moreover, an offline-capable goal-conditioned method — it reports offline D4RL results with that regularizer — so the distinction is not offline versus online. The distinction is the interface: TD InfoNCE learns a raw-goal-conditioned policy and occupancy estimator; it has no reward-prompted latent policy family, no general reward-to-policy-index inference mechanism, and no validated zero-shot control for arbitrary reward functions (its arbitrary-reward evaluation is a fixed-policy appendix observation rather than a deployed mechanism). Density-FB contributes the outcome-space construction, the offline reward-integrating policy extraction (Section 3.10), and the latent task interface and its correction (Section 6). Note also that the headline goal-reaching cells in Table 1 and the latent-index results of Section 6 use the soft variant below; the hard variant — the direct TD InfoNCE correspondence — is the ablation, and it is the stronger configuration for reward transfer (Table 2).

3.6 What changed relative to FB?

Hard Density-FB keeps the forward–backward factorization and the policy-conditioned Bellman bootstrap. The main change is how successor occupancy is represented. Standard FB interprets $F(s,a,z)^\top B(x)$ directly as an unconstrained successor density. Density-FB instead treats the same bilinear quantity as a score and normalizes the scores across sampled successor queries:

$$p_{ij} = \mathrm{softmax}_j \left(F(s_i, a_i, g_i)^\top B(u_j)/\tau \right). \tag{28}$$

This makes the represented object the normalized successor measure $\nu = (1-\gamma)M$, whose sampled Bellman target is

$$p_i^{\mathrm{tgt}} = (1-\gamma)e_i + \gamma \bar{p}_i^{+}. \tag{29}$$

Two estimator choices accompany this change. First, we fit the distributional target with cross-entropy rather than FB's squared/linear density estimator. Second, the query set is built from batch successors, guaranteeing that the observed successor appears in the support of the immediate one-hot target.

We also remove FB's orthonormalization objective. The softmax does not make B orthogonal; rather, $F^\top B$ is no longer an estimate of the density ratio itself. Only the resulting probabilities have predictive meaning, and downstream reward evaluation is performed directly under this distribution. We therefore do not impose an orthonormal constraint on B.

3.7 Why normalization addresses stationary play

Normalization does not delete dwell occupancy. If the cube remains stationary for many transitions, the learned distribution can still assign high probability to that configuration. What it cannot do is increase its total mass:

$$\sum_j p_{ij} = 1. \tag{30}$$

Repeated stationary transitions can therefore change where the predicted occupancy is allocated, but they cannot increase critic scale simply because the same successor appears many times. This preserves the fact that the cube tends to remain where it was left while removing scale as a degree of freedom.

3.8 Soft immediate targets

Hard Density-FB places the immediate $1-\gamma$ mass exactly on the observed successor. We also evaluate a smoothed version.

Define normalized kernel weights

$$k_{ij}^{\epsilon} = \frac{\exp\left(-\frac{1}{2}\|(u_j - x_i^+)/\epsilon\|^2\right)}{\sum_m \exp\left(-\frac{1}{2}\|(u_m - x_i^+)/\epsilon\|^2\right)}. \tag{31}$$

Soft Density-FB changes only the immediate term:

$$p_{i,\text{soft}}^{\text{tgt}} = \text{sg}\left[(1-\gamma)k_i^{\epsilon} + \gamma \bar{p}_i^+\right]. \tag{32}$$

Everything else is unchanged. The hard variant learns the normalized successor measure itself, whereas the soft variant smooths each one-step successor over an ϵ-neighborhood before propagating occupancy. Because the kernel row is normalized over the sampled query set, its effective base measure is the kernel multiplied by the query-pool density, not the kernel alone: the smoothing is data-density weighted and depends on pool composition and size. The hard immediate term is the cleaner estimator in this respect, since the expected one-hot label represents $P_\phi(\cdot \mid s, a)$ directly.

The soft variant is best understood as estimating a different, equally valid successor object rather than as label smoothing. Writing $K_{\rho,\epsilon}(du \mid x) = k_\epsilon(x,u)\,\rho_\phi(du) / \int k_\epsilon(x,v)\,\rho_\phi(dv)$ for the data-density-weighted kernel, the population soft operator is

$$\mathcal{T}_K^\pi \nu = (1-\gamma)\,\mathbb{E}_{x' \sim P_\phi(\cdot|s,a)}[K_{\rho,\epsilon}(\cdot \mid x')] + \gamma\,\mathbb{E}_{s',a'}[\nu(\cdot \mid s', a')], \tag{33}$$

whose fixed point ν_K^π is the kernel-smoothed discounted occupancy. Reward integration under it satisfies $\int r\,d\nu_K^\pi = \int (K_{\rho,\epsilon} r)\,d\nu^\pi$: the soft model evaluates — and its actor optimizes — the smoothed reward $K_{\rho,\epsilon} r$, not r itself. Sums of rewards remain linear under this smoothing, but sharp boundaries, conjunctions, maxima over goals, and avoidance terms are altered; for reward families of that kind the hard variant is the semantically exact interface, and evaluations of them should either use it or state the smoothing.

3.9 Role of the goal

Goal conditioning is not required by the Density-FB estimator. We use g because hindsight relabeling supplies goals from the play data, allowing us to learn a family of goal-conditioned policies $\{\pi_g\}$ and their corresponding successor measures. In this sense, g plays the same indexing role that the latent task vector z plays in standard FB, and training still requires no reward labels or task labels.

3.10 Control

The policy-extraction recipe is unchanged from the corresponding baseline. The only difference is the critic supplied to the actor. A standard critic returns a scalar $Q(s,a,g)$ directly; Density-FB returns a distribution over candidate future configurations, so we first read a scalar goal value from that distribution.

For either hard or soft Density-FB, define the goal readout

$$h_g^{\epsilon}(u_j) \propto \exp\left(-\frac{1}{2}\|(u_j - g)/\epsilon\|^2\right), \qquad \sum_j h_g^{\epsilon}(u_j) = 1. \tag{34}$$

The scalar critic supplied to the actor is

$$q_g(s,a) = \sum_j p_j(s,a,g)\,h_g^{\epsilon}(u_j). \tag{35}$$

The same goal readout is used for both training variants. Hard versus soft refers only to the immediate term of the successor-measure Bellman target, not to the actor or its goal weighting.

The occupancy model and the goal used to train the actor are separate objects. For a goal g, the actor scores actions by the predicted occupancy near the goal, using the same normalized kernel weights as the soft immediate target, evaluated at the goal:

$$q_g(s,a) = \sum_j p_j(s,a,g)\,k_j^{\epsilon}(g), \tag{36}$$

with k^ϵ from Equation 32's kernel and the per-block tolerances for multi-object goals. The kernel weighting is used by both training variants; the hard/soft choice affects only the immediate term of the measure loss, never the actor's goal weighting.

For the flow actor, we use this inside the behavior-regularized objective

$$\mathcal{L}_\pi = \lambda_{\text{BC}} \mathcal{L}_{\text{BC-flow}} - \mathbb{E}\left[\frac{q_g(s, \pi(s, g))}{s_q}\right], \tag{37}$$

where

$$s_q = \mathbb{E}\left[|q_g - \mathbb{E}[q_g]|\right], \qquad \lambda_{\text{BC}} = 3. \tag{38}$$

The predicted occupancy contains a large action-independent component, whereas the actor only needs differences between actions. We therefore center q_g before computing its magnitude normalizer. The behavior-cloning baseline trains the same actor with the q_g term removed. We separately evaluate Gaussian DDPG+BC as an alternative policy extractor.

3.11 The readout is self-normalized importance sampling

Every scalar the actor or the reward evaluation consumes has the same estimator form, and it is worth naming. The model represents $\nu(du \mid s, a) \propto e^{f(s,a,u)}\, \rho_\phi(du)$ with $f = F^\top B/\tau$, so for queries $u_j \sim \rho_\phi$ any finite-pool readout

$$\sum_j p_j(s, a)\, r(u_j) = \frac{\sum_j e^{f(s,a,u_j)}\, r(u_j)}{\sum_j e^{f(s,a,u_j)}} \tag{39}$$

is self-normalized importance sampling of $\mathbb{E}_\nu[r]$, with proposal ρ_ϕ and unnormalized weights e^f (Owen, 2013). Three consequences follow. First, the estimator is consistent as the pool grows, under the usual support and moment conditions, but biased at any finite pool size, as every self-normalized estimator is; this, not implementation detail, is why the open items call for pool-size curves (Section 7). Second, the standard quality diagnostic for such weights is their effective sample size (Kong, 1992), which we monitor throughout training as a collapse check on the learned weights. Third, the zero-shot reward evaluation of Section 5 is the same estimator class evaluated on a larger pool, so its finite-pool correction inherits the same analysis.[1] The softmax weights therefore play two roles at once: they are the learned successor probabilities, and they are the importance weights of the readout. This is why the composition of the query support — the data-density base measure discussed in Section 3.8 — enters the semantics of every readout, not only the soft training target.

3.12 One training step

Algorithm 1 summarizes one Density-FB update.

4 Goal-conditioned results

We evaluate on state-based OGBench manipulation and locomotion (Park et al., 2025). All of our models are trained for 500k gradient steps with no reward labels. Native evaluation uses five tasks, 50 episodes each, and three seeds; intervals are 95% t-based confidence intervals.

We evaluate each representation with two actor extractors, FQL + BC and Gaussian DDPG+BC, to separate representation quality from policy extraction. Behavior cloning trains the same actor with the q_g term removed and learns no successor representation: it reaches 0.111 on cube-single and 0.019 on cube-double. Published GCIQL results use 1M gradient steps and eight seeds. We include them as an external reference rather than a matched baseline.

[1] One implementation detail qualifies the statement. F is a two-head ensemble, and both the actor and the shipped evaluation take the elementwise minimum of the two heads' softmax weight vectors before the readout, without renormalizing. The resulting weights are conservative but sum to at most one, so what is computed is a sub-normalized version of Equation 39; the consistency statement above is for the single-head weights. Every cell of Table 2 uses this two-head path.

Algorithm 1 Density-FB, one gradient step. Blue marks changes to the successor-measure estimator relative to standard FB. Teal/orange marks the hard/soft immediate-target choice. Goal conditioning, the choice of ϕ, and policy extraction are experimental setup choices and are left black.

Sample transitions $\{(s_i, a_i, s_i^+)\}_{i=1}^n$ and hindsight goals $\{g_i\}_{i=1}^n$
Set $x_i^+ = \phi(s_i^+)$
Build the query support $U = \{u_j = x_j^+\}_{j=1}^n$ ▷ FB samples queries $x \sim \rho$
Compute bilinear scores $c_{ij} = F(s_i, a_i, g_i)^\top B(u_j)$
Normalize $p_{ij} = \mathrm{softmax}_j(c_{ij}/\tau)$ ▷ FB treats $F^\top B$ as a density
Sample bootstrap actions $a_i^+ \sim \pi_{\bar{\omega}}(\cdot \mid s_i^+, g_i)$
Evaluate target-network occupancies $\bar{p}_i^+$ over the same query support U
Hard: $\eta_i = e_i$, where $e_{ij} = \mathbf{1}[j = i]$ ▷ Soft: $\eta_i = k_i^\epsilon$
Form $p_i^{\mathrm{tgt}} = \mathrm{sg}\big[(1-\gamma)\eta_i + \gamma \bar{p}_i^+\big]$ ▷ normalized Bellman target
Update F, B with $-\frac{1}{n}\sum_i \sum_j p_{ij}^{\mathrm{tgt}} \log p_{ij}$ ▷ FB: squared/linear loss + ortho
Update the actor using q_g from Equation 36, with F, B frozen ▷ same extractor as baseline
Soft-update the target networks

Environment	Actor	Plain FB	Density-FB (soft)	Density-FB (hard)	GC-IQL (published)
cube-single	FQL + BC	0.420 (10 seeds)	**0.929 ± 0.085**	0.915 ± 0.087	–
	DDPG + BC	0.353 ± 0.38	0.879 ± 0.095	0.887 ± 0.066	0.68 ± 0.06
cube-double	FQL + BC	0.005 ± 0.016	**0.736 ± 0.139**	0.680 ± 0.070	–
	DDPG + BC	0.013 ± 0.029	**0.646 ± 0.019**	0.599 ± 0.077	0.40 ± 0.05
scene	FQL + BC	0.076 ± 0.057	0.628 ± 0.134	**0.641 ± 0.109**	–
	DDPG + BC	0.047 ± 0.057	0.563 ± 0.011	0.581 ± 0.322	0.51 ± 0.04
antmaze-medium	FQL + BC	0.593 ± 0.25	0.563 ± 0.167	0.669 ± 0.240	–
	DDPG + BC	pending	0.829 ± 0.058	**0.877 ± 0.097**	0.71 ± 0.04

Table 1: Native goal-conditioned success by environment and policy-extraction rule. All Density-FB cells use three seeds except cube-double with DDPG + BC, which uses four. Plain FB uses three seeds except cube-double with FQL + BC (four) and scene with FQL + BC (five). The pending cell never ran: its launch failed because the AntMaze dataset was absent from the expected path. Seeds for the completed manipulation cells: soft flow cube-single .896/.928/.964, cube-double .680/.736/.792, scene .672/.568/.644; hard flow cube-single .876/.944/.924, cube-double .692/.700/.648, scene .692/.620/.612; soft DDPG + BC cube-double .644/.660/.648/.632, hard DDPG + BC cube-double .668/.560/.596/.572.

On cube-single, both Density-FB variants reach about 0.92, leaving little room to distinguish them. The harder manipulation tasks separate the methods more clearly. On cube-double, soft Density-FB reaches 0.736 with the flow actor, compared with 0.005 for plain FB and 0.019 for behavior cloning; with DDPG+BC it reaches 0.646, against a published 0.40 for GCIQL at twice the training budget. On scene, Density-FB reaches about 0.64 while plain FB remains near zero.

The scene average also hides a consistent task split. All seeds solve open, unlock-and-lock, and rearrange-medium (0.88–1.0). put-in-drawer (0.02–0.50) and rearrange-hard (0.0–0.04) remain difficult.

Manipulation results are similar under DDPG+BC. Switching from FQL + BC to DDPG+BC lowers soft Density-FB from 0.929 to 0.879 on cube-single and from 0.736 to 0.646 on cube-double. The improvement is therefore not specific to the flow actor. On antmaze-medium, the same switch moves Density-FB from 0.563 to 0.829, so actor extraction matters much more there.

Density-FB does not improve AntMaze. antmaze-medium removes the structure that motivated Density-FB: the goal-relevant variable is the agent itself, so there is no manipulated object that remains stationary while actions continue. Under DDPG+BC, Density-FB reaches 0.829 against 0.837 for our matched GCIQL control, and both remain below the strongest published method on that benchmark (0.96 for HIQL). This supports the narrower hypothesis that Density-FB helps when play data contains long stationary segments in the goal-relevant object variables.

Isolating the estimator from the outcome coordinates. Density-FB departs from FB in two places at once: the categorical estimator, and the privileged outcome coordinates ϕ that the successor measure is predicted over. Two controls separate them on cube-double, the discriminating environment, matched in everything else (one seed each, 500k steps; we quote the 500k value and, in parentheses, the mean over evaluations after 350k). A paper-exact TD InfoNCE critic (Proposition 1) reaches 0.576 (0.545). A goal-conditioned outcome-space LSIF model — FB's own unnormalized loss, but on our ϕ, with our external kernel readout and our actor — reaches 0.300 (0.329). Both fall below soft Density-FB (0.736) and hard Density-FB (0.680). Read together they split the improvement: ϕ plus an external readout carries plain FB from 0.005 to 0.300, and the normalized categorical estimator carries 0.300 to 0.736. The privileged coordinates are therefore load-bearing, and the estimator change still accounts for the larger share. These are single-seed runs, and the matched-extractor TD InfoNCE arm has not been run (Section 7).

5 Reward evaluation

Goal reaching only tests whether the learned measure supports the benchmark goals. Reward transfer asks whether the same measure can evaluate a reward that was never used for training.

At test time, we draw a pool $U_P = \{u_j\}_{j=1}^{16,384}$ of object configurations from the dataset. For a given (s, a, g), we compute the normalized occupancy $p(s, a, g; U_P)$ over this pool and evaluate the reward once on every pool element. The direct Monte-Carlo readout is

$$\frac{p(s, a, g; U_P)^\top r(U_P)}{1 - \gamma}. \tag{40}$$

Because stationary transitions concentrate predicted occupancy near the current object configuration, finite-pool smoothing matters most there. We therefore apply a local correction at $x = \phi(s)$.

Let $c_x \in \Delta^{|U_P|-1}$ denote the normalized kernel weights of x over the pool (the same k^ϵ as Equation 36). We replace the interpolated reward $c_x^\top r(U_P)$ with its exact value $r(x)$:

$$\widehat{Q}_r(s, a, g) = \frac{r(x) + \left(p(s, a, g; U_P) - c_x\right)^\top r(U_P)}{1 - \gamma}. \tag{41}$$

The correction is action independent, so it changes the absolute value estimate without changing the ranking of actions. The resulting readout remains linear in r and requires no retraining.

To act under a new reward, we score candidate goals using Equation 41, select the highest-valued goal from the initial state, and execute its goal-conditioned policy. This replaces FB's analytic reward-to-z mapping with search over the learned goal-conditioned policy family.

Reward construction. We test three reward families, all defined only from object coordinates, valued in $[0, 1]$, and never observed during training (Figure 1; internal names dirac/dense/region):

sparse, the benchmark tolerance indicator

$$r(x) = \mathbf{1}[\|x - g\| \le \epsilon], \tag{42}$$

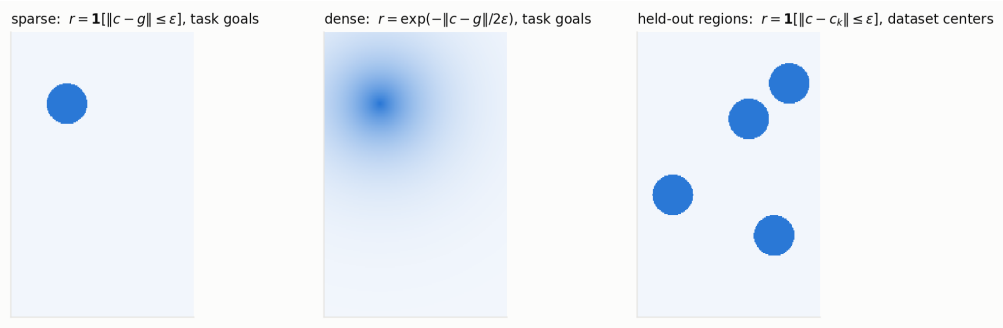


Figure 1: The three test reward families on the cube workspace (cube-position plane, ϵ-balls to scale). Sparse and dense share task-style centers and differ in shaping; held-out regions keep the sparse form at hindsight-drawn dataset centers.

conjunctive over object blocks, at the five task goals;

dense, the same centers with distance-based shaping

$$r(x) = \exp(-d(x, g)/2\epsilon) \tag{43}$$

in tolerance-scaled coordinates; and held-out regions, the same sparse indicator centered at reachable dataset configurations that are not benchmark goals. These centers are drawn by hindsight from within episodes and kept only when at least five relabeled samples fall inside the region. Dense versus sparse varies reward shaping while keeping the goal fixed. Sparse versus held-out regions keeps the reward form fixed while changing where in object space the reward is located. We evaluate 25 episodes per task and three seeds. The fixed-partition ablation replaces the resampled query set with 256 fixed prototypes while leaving the rest of the objective unchanged.

Environment	Reward family	Plain FB	Density-FB (soft)	Density-FB (hard)
cube-single	sparse	0.413 ± 0.308	0.816 ± 0.269	$\mathbf{0.888 \pm 0.091}$
	dense	0.488 ± 0.397	0.725 ± 0.301	$\mathbf{0.912 \pm 0.087}$
	held-out regions	0.090 ± 0.114	0.293 ± 0.052	$\mathbf{0.647 \pm 0.301}$
cube-double	sparse	0.013 ± 0.023	0.549 ± 0.240	$\mathbf{0.573 \pm 0.126}$
	dense	0.037 ± 0.011	0.488 ± 0.163	$\mathbf{0.520 \pm 0.121}$
	held-out regions	0.000 ± 0.000	0.177 ± 0.160	0.167 ± 0.014
scene	sparse	0.472 ± 0.124	0.395 ± 0.092	$\mathbf{0.640 \pm 0.111}$
	dense	0.413 ± 0.030	0.347 ± 0.219	$\mathbf{0.627 \pm 0.094}$
	held-out regions	0.053 ± 0.080	0.220 ± 0.086	$\mathbf{0.347 \pm 0.038}$

Table 2: Zero-shot reward transfer (mean $\pm$ 95% CI, three seeds unless marked). GCIQL has no reward readout of this kind and is therefore absent. Plain FB is evaluated through its own reward interface, $z_r \propto \mathbb{E}_\rho[r(x)B(x)]$ followed by rollout under z_r, since it has no density readout; the reward families, pool, episode count, and success criterion are identical across all three columns, so only the readout differs. All cells now use three seeds; the plain-FB seeds missing at first submission were trained to parity. The plain-FB cube-double entries sit at the floor rather than merely low: they are 1 and 0 successes in 125 episodes, against a random-index floor of 0.012 and 0.000. All numbers use a 16,384-configuration pool and 25 episodes per task.

The largest difference appears on rewards centered away from the benchmark goals. On cube-single, held-out-region transfer rises from 0.133 with the fixed partition to 0.293 with soft Density-FB and 0.647 with the hard variant (Table 3). This is consistent with continuous query representations helping most when the reward sits between or away from fixed prototypes.

cube-single ablation	sparse	dense	held-out regions
Density-FB (soft)	0.816 ± 0.269	0.725 ± 0.301	0.293 ± 0.052
Fixed partition (K=256)	0.725 ± 0.121	0.723 ± 0.298	0.133 ± 0.201

Table 3: Replacing the resampled query set with 256 fixed prototypes at otherwise identical settings. The largest drop occurs on held-out regions.

Hard and soft targets behave differently across environments. On cube-single and scene, the hard target improves all three reward families; on scene, for example, sparse transfer rises from 0.395 to 0.640. On cube-double, however, the two targets sit inside each other's intervals (0.549 vs. 0.573, 0.488 vs. 0.520, 0.177 vs. 0.167). Hard targets therefore improve reward readout on some manipulation tasks and not others. On goal reaching, the two variants are within noise everywhere (Table 1).

Comparison with published zero-shot methods. Table 4 places our sparse-reward results alongside published state-based OGBench results from Bagatella et al. (2026a). This is not a controlled comparison. The published results use 1M gradient steps, 10 seeds, and 10 evaluation episodes per task, whereas ours use 500k steps, three seeds, and 25 episodes. The task-index inference also differs: their methods use linear regression on relabeled data, whereas Density-FB searches over candidate goals. Our matched plain-FB column partially validates itself against that table: it reproduces the published FB result on cube-single (0.536 on the stock-hyperparameter seed, 0.413 ± 0.308 over the three seeds of Table 2, against 49.6 published) and the cube-double floor ($\leq$0.056 vs 2.6), but on scene our FB reaches $\sim$0.45 against a published 12.8, so the published scene FB number should not be used as a comparison point for this stack; the matched local baseline is the honest one. A related observation from the same probes: on scene, plain FB's reward interface outperforms its own goal conditioning (0.45 via z_r vs 0.14 native), so FB's weakness there is its goal encoding rather than its reward inference.

Method	cube-single	cube-double	scene
Laplacian (Wu et al., 2018)	15.1 ± 1.5	2.0 ± 0.4	7.8 ± 1.3
RLDP (Jajoo et al., 2026)	19.8 ± 2.4	3.8 ± 0.8	11.6 ± 1.6
ICVF (Ghosh et al., 2023)	20.4 ± 1.9	5.0 ± 0.8	45.4 ± 2.3
BYOL	22.0 ± 3.2	4.4 ± 0.7	15.4 ± 1.4
TD-JEPA (Bagatella et al., 2026a)	34.2 ± 2.9	3.6 ± 0.8	38.4 ± 1.4
FB (Touati & Ollivier, 2021)	49.6 ± 3.8	2.6 ± 0.4	12.8 ± 1.6
HILP (Park et al., 2024)	74.2 ± 3.5	20.0 ± 2.7	43.8 ± 1.9
BYOL-γ	79.4 ± 2.8	2.6 ± 0.7	14.4 ± 2.3
Density-FB (soft)	81.6 ± 26.9	$\mathbf{54.9 \pm 24.0}$	39.5 ± 9.2
Density-FB (hard)	$\mathbf{88.8 \pm 9.1}$	$\mathbf{57.3 \pm 12.6}$	$\mathbf{64.0 \pm 11.1}$

Table 4: Zero-shot success rate (%) on state-based OGBench. Rows above the line are as reported by Bagatella et al. (2026a) at 1M gradient steps, 10 seeds and 10 episodes per task, with the task index inferred by linear regression. Rows below are our sparse reward family at 500k steps, three seeds and 25 episodes, with the index chosen by search. Our intervals are t-based on three seeds.

The largest numerical gap is on cube-double: the strongest published result is 20.0, while both Density-FB variants exceed 54. On scene, hard Density-FB reaches 64.0 versus 45.4 for the strongest published result. The cube-single comparison is less informative: hard Density-FB reaches 88.8 versus 79.4, but the difference lies within our three-seed confidence interval. These comparisons remain suggestive until the protocols are matched.

cube-double is also where the published methods are closest to each other and to zero: six of the eight fall below 5.

Reward transfer remains harder than direct goal reaching. On cube-double, the same soft checkpoint reaches 0.736 under ordinary goal conditioning and 0.549 when the same task is

specified through the sparse reward readout. Because the checkpoint is identical, this gap enters through the reward-readout and goal-selection pipeline rather than through representation training. An oracle that selects the reward center directly would separate readout error from goal-selection error.

6 A latent task index

Everything above conditions on a goal g. Standard FB instead conditions on a latent vector z, which is what makes its zero-shot interface possible: any reward is embedded as $z_r \propto \mathbb{E}_\rho[r(x)B(x)]$ and executed by the policy indexed by z_r. This section ports that interface to Density-FB, shows where the direct port fails, and gives the correction.

6.1 The direct port fails

The natural encoding replaces the goal with its backward embedding,

$$z(g) = \sqrt{d}\,\frac{\bar{B}(\phi(g))}{\|\bar{B}(\phi(g))\|}, \tag{44}$$

trains the measure and actor on a mixture of hindsight indices $z(g)$ and random unit-sphere indices, and replaces the actor's kernel readout with the latent reward $r_z(u) = B(u)^\top z/\sqrt{d}$. This collapses. On cube-double, success falls from 0.736 (goal-conditioned) to 0.017, at the behavior-cloning floor of 0.019; on cube-single the corresponding arms read 0.096–0.108 against a floor of 0.111. The failure is not in the estimator: under z the measure trains to the same cross-entropy as under g and shows the same occupancy-ranking fidelity against Monte-Carlo rollouts.

6.2 Diagnosis: the backward basis has an arbitrary, gauge-dominated geometry

The cross-entropy loss constrains only the product $F^\top B$: for any invertible C, replacing B by CB and F by $C^{-\top}F$ leaves every prediction unchanged. The softmax adds a further exact invariance: for any constant vector b, replacing $B(x)$ by $B(x) + b$ shifts every logit in a row by the same $F^\top b$, which the softmax cancels. The mean of B is therefore pure gauge — it can grow without changing a single predicted probability — and training drives it to an extreme value. Measured on every checkpoint, $\|\mu_B\|$ is 33–103 times the average centered feature norm $\mathbb{E}\|B - \mu_B\|$, so the uncentered second moment $\Sigma_B = \mathbb{E}_\rho[B(x)B(x)^\top]$ is dominated by a single gauge direction: effective rank 1.0002 of $d = 50$, condition number 10^6–10^7. The centered covariance tells the opposite story: $\mathrm{Cov}(B)$ has effective rank 6.3–20.8 of 50 across the same checkpoints. The informative geometry is rich; what looks like rank collapse is a softmax-gauge mean sitting on top of it. Two consequences follow.

First, Equation 44 collapses. Normalizing $B(g)$ pins every goal to the gauge direction: over 4,096 dataset goals, the mean pairwise cosine of $z(g)$ is 0.998–1.000, where matched random unit vectors give 0.002; subtracting the mean before normalizing drops it to ∼0.003. The actor behaves accordingly: its output moves by 0.000 when the goal index changes (noise floor 0.19), while a random index moves it by 1.9–2.0. The policy is index-capable; the encoding never hands it two different indices. The same collapse appears on goal-conditioned checkpoints that never trained with z, so it is a property of the encoding, not of latent training.

Second, FB's reward embedding is the wrong vector once the measure is normalized. The identity behind $z_r \propto \mathbb{E}[rB]$ is linear — $Q_r = F^\top \mathbb{E}[rB]$ holds because the FB value is linear in B — and the softmax removes exactly that linearity. To read a reward through $r(u) \approx B(u)^\top c$, the correct coefficient is the least-squares one, $c_r = (\Sigma_B + \lambda I)^{-1}\mathbb{E}[rB]$; the moment $\mathbb{E}[rB]$ coincides with it only when $\Sigma_B \approx I$, which is what standard FB's orthonormality term enforces and Density-FB deliberately removed. With Σ_B gauge-dominated, the two are nearly orthogonal: measured $\cos(\mathbb{E}[rB], c_r) = 0.010$–$0.025$. The consequences are direct. Reconstructing goal rewards as $B(u)^\top z$ scores AUC 0.43–0.87 when z is the moment or the point embedding of Equation 44, and 0.99–1.00 when $z = c_r$. Assigning rewards to their indices is at chance with the moment (0.35–0.51 coordinate percentile, where

0.50 is chance) and matches the goal-indexed control with c_r (0.000–0.008). Consistent with the gauge diagnosis, centering alone recovers most of this: the centered point index normalize$(B(g) - \mu_B)$ reconstructs rewards at AUC 0.99–1.00 on the same pools (verified on disjoint fit and evaluation halves), while the inverse covariance additionally expands low-variance task directions and spreads the index set (participation ratio 28–30 versus 6.6–19.8 for centered pointing).

The uncentered encoder also fails an invariance a task code must have. Adding a constant β to a reward leaves its optimal policy unchanged but shifts the moment by $\beta\,\mathbb{E}[B]$ — almost exactly the gauge direction — and measured on our checkpoints the uncentered index of r and of $r + 1$ are nearly orthogonal ($\cos = 0.035$, -0.000, -0.044 on the three checkpoints tested), while the centered construction below is invariant to the digit.

6.3 The corrected index

The repair follows from the diagnosis: encode tasks with the least-squares coefficient rather than the moment, and let the actor integrate the actual reward rather than a latent surrogate. Given a task specified by reward samples $\{(u_j, r_j)\}_{j=1}^{M}$,

$$\hat{z} = \sqrt{d}\,\frac{(\hat{\Sigma}_B + \lambda I)^{-1}\hat{m}}{\left\|(\hat{\Sigma}_B + \lambda I)^{-1}\hat{m}\right\|}, \qquad \hat{m} = \frac{1}{M}\sum_j \bar{B}(u_j)\, r_j, \tag{45}$$

where $\hat{\Sigma}_B$ is an exponential moving average of $\bar{B}(u)\bar{B}(u)^\top$ over query samples, stored with the network parameters so that training and evaluation whiten with the same statistics. Hindsight goals supply reward samples for free through the kernel of Equation 36, $r_j = k_j^{\epsilon}(g)$. The measure and the actor condition on $\hat{z}$; the actor's scalar critic integrates the reward samples themselves,

$$q_{\hat{z}}(s, a) = \sum_j p_j(s, a, \hat{z})\, r(u_j), \tag{46}$$

so the latent serves only as a policy index and the reward semantics stay external — the same division of labor that makes the goal-conditioned readout work. This is whitening at the interface rather than in the loss: the orthonormality penalty that would make $\mathbb{E}[rB]$ correct also caps manipulation success at 0.225 (Section 7), so the basis is left free and the correction is applied where the index is built.

The gauge analysis identifies one further correction, which the first corrected-index run predates and the centered arms of Section 6 implement. Because the mean of B is unidentifiable and reward shifts move the moment along it, the principled statistics are the centered ones:

$$\hat{z}^{\,\mathrm{c}} = \sqrt{d}\,\frac{(\hat{C}_B + \lambda I)^{-1}\hat{m}^{\mathrm{c}}}{\left\|(\hat{C}_B + \lambda I)^{-1}\hat{m}^{\mathrm{c}}\right\|}, \qquad \hat{m}^{\mathrm{c}} = \frac{1}{M}\sum_j \left(\bar{B}(u_j) - \hat{\mu}_B\right)(r_j - \bar{r})\,, \tag{47}$$

with $\hat{\mu}_B$ and $\hat{C}_B$ the running mean and centered covariance and $\bar{r}$ the sample-mean reward.

Proposition 2 (Invariances of the centered index). $\hat{z}^{\,\mathrm{c}}$ is invariant to the gauge translation $B(x) \mapsto B(x) + b$, to the reward shift $r \mapsto r + \beta$, and, after normalization, to positive reward scaling $r \mapsto \alpha r$. The uncentered index of Equation 45 has only the last of these.

Proof sketch. Centering removes both translations exactly: $(B + b) - (\mu_B + b) = B - \mu_B$ and $(r + \beta) - (\bar{r} + \beta) = r - \bar{r}$, and $\hat{C}_B$ is unchanged by either; scaling r by $\alpha > 0$ scales $\hat{m}^{\mathrm{c}}$ linearly and is absorbed by the normalization. □

Two scope notes on this proposition. First, it corrects the additive gauge only: the factorization also admits $B \mapsto AB$, $F \mapsto A^{-\top}F$ for invertible A, under which the coefficient transforms dually ($c \mapsto A^{-\top}c$ at $\lambda = 0$) but isotropic ridge regularization and sphere normalization are not invariant, so the encoder is translation-gauge-invariant, not fully gauge-invariant. The additive direction is the empirically dominant pathology. Second, the invariance is exact for the population object; the implementation tracks $\hat{\mu}_B$ and $\hat{C}_B$ as moving averages during joint training, and only approximates it (both moments run on the same schedule against the same base measure — the batch next-state marginal used for queries —

with $\hat{C}_B$ symmetrized and eigenvalue-floored, and the mismatch between stored and probe statistics logged).

The centered encoder also forces an interface split that the uncentered form obscured. Its reward-sample moment vanishes identically at $M = 1$ ($r_1 - \bar{r} = 0$), so it has no one-sample limit: point-goal prompting and reward-sample prompting are different inference modes. For goals we use the analytic centered point prompt $z_g = \sqrt{d}\ \mathrm{normalize}(\bar{B}(\phi(g)) - \hat{\mu}_B)$ — the population moment of a Dirac-style goal reward — while the general interface consumes an actual sample set with $M > 1$ (in practice $M \geq 256$), and results for one mode are not evidence for the other. A constant reward has $\hat{m}^c = 0$ exactly; the implementation returns a neutral index when $\|(\hat{C}_B + \lambda I)^{-1}\hat{m}^c\|$ falls below a threshold, since normalizing a near-zero coefficient would amplify noise.

6.4 Result

Both links of the repair were verified on existing checkpoints before retraining: the ridge solve de-collapses the index (participation ratio 1.0 → 9–38), and the least-squares coefficient fixes reward reconstruction and assignment as quoted above. We then retrained with the corrected index (cube-single, 500k steps, one seed per arm), in both encodings and in both inference modes that the centering forces apart (see the discussion after Proposition 2). The analytic point prompt encodes a goal as the population moment of a Dirac-style goal reward, $\hat{z} \propto (\hat{\Sigma}_B + \lambda I)^{-1}\bar{B}(\phi(g))$; the reward-sample prompt is the general interface, and builds $\hat{z}$ from an actual set of $M = 256$ samples $\{(u_j, r_j)\}$ drawn from a frozen pool, with no one-sample shortcut available to it. Evaluation is noisy at this budget, so we report the 500k value and the mean over evaluations after 350k:

Index	Prompt at evaluation	at 500k	mean ≥ 350k
centered, Eq. 47	reward samples, M=256	**0.816**	0.785
centered, Eq. 47	analytic point	0.824	0.868
uncentered, Eq. 45	analytic point	0.812	0.743
uncentered, actor uses $B^\top \hat{z}$	analytic point	0.368	0.291
uncorrected latent arms	—	0.096–0.108	—
behavior cloning	—	0.111	—
goal-conditioned Density-FB (soft)	goal	0.929	—

The corrected index with the sample-based readout is the first latent-conditioned Density-FB that works: roughly 7.5× the uncorrected arms and 88% of the goal-conditioned anchor. Three readings follow from the rows. First, the top row is the general interface actually exercised: that number comes from a rollout whose index was built from 256 reward samples, not from the point shortcut, and it matches the point prompt on the same encoder (0.816 against 0.824), so for goal-kernel rewards the two inference modes agree. Second, centering costs nothing: the centered arms reach 0.824 and 0.816 against 0.812 for the uncentered encoder, so the invariances of Proposition 2 are bought for free, and we take the centered index of Equation 47 as the method and keep the uncentered run as an ablation. Third, the gap between the last two rows isolates the readout: conditioning is identical, so integrating real reward samples versus the linear surrogate $B^\top \hat{z}$ accounts for 0.812 versus 0.368. During training the index stays de-collapsed, and on the uncentered run the actor's index sensitivity rises from exactly 0.000 to 0.132 (measured at 100k steps), with the earlier pathology inverted: goal-derived indices now move the policy more than random ones (0.132 vs 0.065).

Three limits, stated plainly. These are single-seed numbers on one environment; the multi-seed and cube-double runs have not been done. Evaluation noise is real — the uncentered run read 0.564 at 400k, and the centered point arm averages 0.868 after 350k against 0.824 at the endpoint — which is why the time-averaged column is quoted alongside the 500k value. And the rewards driving the sample interface so far are goal kernels: the mechanism is exercised, but the property that would make this a zero-shot reward interface — transfer to reward families the index has never seen, including the constant-shifted copies that Proposition 2 is about — is still pending (Section 7). These numbers come from different arms than the uncorrected ones and are not comparable to them beyond the floor-versus-working contrast.

7 Open items

The following are not yet done, and the claims above should be read with them in mind.

Matched-protocol baselines. Every published number in Table 4 is quoted rather than re-run. A controlled comparison requires rerunning these methods with the same training budget, seeds, evaluation episodes, and task-index inference. Until that is done, the table shows our numbers next to published ones rather than a controlled comparison. We have re-run FB from the same codebase on cube-single; the remaining environments and methods have not yet been checked this way. The contemporary obligations are TD-JEPA (Bagatella et al., 2026a), which evaluates on OGBench manipulation, and distribution-corrected FB (Asadulaev et al., 2026), which evaluates on the same cube environments. The estimator-level control is TD InfoNCE (Proposition 1), and it requires two arms rather than one: a paper-exact replication (ℓ_2-normalized towers, its query supports, its goal-contrastive actor with behavioral-cloning regularization and goal relabeling), which measures the published method in our setting, and a matched-extractor variant (its critic under our outcome coordinates, external reward readout, and actor), which separates the estimator from the extraction. The paper-exact arm has been run, together with the outcome-space LSIF control of Section 2 — $F(s,a,g)^\top B(\phi(x))$ with the unnormalized FB loss but our ϕ, our external kernel readout, and our actor, which isolates the estimator change from the privileged outcome coordinates; both are reported in Section 4 at one seed on cube-double. Outstanding here are the matched-extractor TD InfoNCE arm, additional seeds for both controls, and the remaining environments. Standard full-observation FB remains in the tables separately as the literature method.

Seed parity. Our cells use three seeds against ten in the published results, which is why several of our confidence intervals are wider than the differences they bracket. The cube-single comparison in Table 4 lies inside our interval and should not be read as a separation until the seed count matches.

Cells still outstanding. One table cell is missing: plain FB with the DDPG+BC actor on antmaze-medium, whose launch failed because the AntMaze dataset was absent from the expected path. This value is already established in the reference FB codebase, so the cell will be filled from that source with its provenance cited, rather than re-run here.

Latent-index coverage. The corrected-index results of Section 6 are one seed per arm on cube-single, and the rewards driving the sample interface there are goal kernels at a single pool size. Outstanding: additional seeds, cube-double, and the reward-family transfer evaluation through the same interface — the direct analogue of Table 2 with $\hat{z}$ in place of g, on the centered encoder of Equation 47, with constant-shifted copies of each reward family included as the affine-invariance check of Proposition 2, and reported over reward-sample pool sizes M rather than at $M = 256$ alone (these curves also quantify the self-normalized-importance-sampling bias of Section 3.11). That evaluation should be accompanied by a search-over-z oracle — optimizing the index directly against the model's own reward integral — since it separates a failing task encoder (closed-form fails, search succeeds) from a policy family that lacks the required behavior (both fail while reward reconstruction is good). All of it is evaluation-only on the checkpoint already trained. Until it runs, Section 6 claims a repaired index driven by real reward samples, not a validated zero-shot reward interface.

Full-state successor ablation. Our main manipulation experiments predict occupancy over object coordinates, $x = \phi(s)$. We are also testing $\phi(s) = s$, where the backward representation receives the full next state. This will separate the effect of normalized successor learning from the choice to predict only the goal-relevant state factor.

Separating normalization from the fitting loss. The current method combines a softmax-normalized successor representation with a cross-entropy Bellman loss. A useful ablation is to keep the same normalized Bellman target

$$p_i^{\mathrm{tgt}} = (1-\gamma)e_i + \gamma \bar{p}_i^{+} \tag{48}$$

but fit it with squared error instead of cross-entropy. If both objectives behave similarly, the main effect comes from normalization; if not, the fitting loss is part of the improvement.

Orthonormality. Equation 5 includes the orthonormality term used by standard FB; Density-FB removes it entirely. No orthonormality, mass, or non-negativity penalty appears anywhere in the implementation (the configuration layer refuses them), because the softmax supplies the normalization those terms approximate. This is deliberate: with a frozen capable representation, orthonormality bounds the critic scale but caps success at 0.225.

Separating readout error from goal-selection error. The gap between goal reaching and reward transfer on cube-double currently combines errors from reward integration and goal selection. Comparing the retrieved goal against an oracle that selects the reward center directly would separate the two.

References

Arip Asadulaev, Maksim Bobrin, Salem Lahlou, Dmitry Dylov, Fakhri Karray, and Martin Takac. Zero-shot off-policy learning. arXiv preprint arXiv:2602.01962, 2026. citation details unverified.

M. Bagatella, M. Pirotta, A. Touati, A. Lazaric, and A. Tirinzoni. TD-JEPA: Latent-predictive representations for zero-shot reinforcement learning. In The Fourteenth International Conference on Learning Representations, 2026a.

Marco Bagatella, Thomas Rupf, Georg Martius, and Andreas Krause. Soft forward-backward representations for zero-shot reinforcement learning with general utilities. arXiv preprint arXiv:2602.06769, 2026b.

L. Blier, C. Tallec, and Y. Ollivier. Learning successor states and goal-dependent values: A mathematical viewpoint. arXiv preprint arXiv:2101.07123, 2021.

Edoardo Cetin, Ahmed Touati, and Yann Ollivier. Finer behavioral foundation models via auto-regressive features and advantage weighting. arXiv preprint arXiv:2412.04368, 2024.

Peter Dayan. Improving generalization for temporal difference learning: The successor representation. Neural Computation, 1993.

Jesse Farebrother, Matteo Pirotta, Andrea Tirinzoni, Rémi Munos, Alessandro Lazaric, and Ahmed Touati. Temporal difference flows. In International Conference on Machine Learning, 2025. arXiv:2503.09817.

Kevin Frans, Seohong Park, Pieter Abbeel, and Sergey Levine. Unsupervised zero-shot reinforcement learning via functional reward encodings. In International Conference on Machine Learning, 2024. arXiv:2402.17135.

Dibya Ghosh, Chethan Anand Bhateja, and Sergey Levine. Reinforcement learning from passive data via latent intentions. In International Conference on Machine Learning, pp. 11321–11339. PMLR, 2023.

P. Jajoo, H. Sikchi, S. Agarwal, A. Zhang, S. Niekum, and M. White. Regularized latent dynamics prediction is a strong baseline for behavioral foundation models. In The Fourteenth International Conference on Learning Representations, 2026.

Scott Jeen, Tom Bewley, and Jonathan M. Cullen. Zero-shot reinforcement learning from low quality data. In Advances in Neural Information Processing Systems, 2024. arXiv:2309.15178.

Augustine Kong. A note on importance sampling using standardized weights. Technical Report 348, University of Chicago, Department of Statistics, 1992.

Art B. Owen. Monte Carlo Theory, Methods and Examples. 2013. Available at https://artowen.su.domains/mc/.

S. Park, T. Kreiman, and S. Levine. Foundation policies with hilbert representations. In The Twelfth International Conference on Learning Representations (ICLR'24). ICLR, 2024. Published online: iclr.cc.

Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. Ogbench: Benchmarking offline goal-conditioned reinforcement learning. In International Conference on Learning Representations (ICLR), 2025.

Thomas Rupf, Marco Bagatella, Marin Vlastelica, and Andreas Krause. Optimistic task inference for behavior foundation models. In International Conference on Learning Representations, 2026. arXiv:2510.20264.

Dmitry Shribak et al. Bellman diffusion models. arXiv preprint arXiv:2407.12163, 2024. citation details unverified.

Bofu Sun et al. Unsupervised zero-shot reinforcement learning via dual-value forward-backward representation. In International Conference on Learning Representations, 2025. OpenReview 0QnKnt411O; citation details unverified.

Ahmed Touati and Yann Ollivier. Learning one representation to optimize all rewards. In Advances in Neural Information Processing Systems (NeurIPS), 2021.

Ahmed Touati, Jérémy Rapin, and Yann Ollivier. Does zero-shot reinforcement learning exist? In International Conference on Learning Representations (ICLR), 2023.

Yifan Wu, George Tucker, and Ofir Nachum. The laplacian in rl: Learning representations with efficient approximations. arXiv preprint arXiv:1810.04586, 2018.

Chongyi Zheng, Ruslan Salakhutdinov, and Benjamin Eysenbach. Contrastive difference predictive coding. In International Conference on Learning Representations, 2024. arXiv:2310.20141.

Chongyi Zheng, Rahul Jayanth, and Benjamin Eysenbach. Can we really learn one representation to optimize all rewards? arXiv preprint arXiv:2602.11399, 2026a. citation details unverified.

Chongyi Zheng et al. Intention-conditioned flow occupancy models. In International Conference on Learning Representations, 2026b. arXiv:2506.08902; citation details unverified.